MediSeba - Intelligent Healthcare Appointment & Prescription Platform

Submitted By

Student Name	Student ID
Md. Emon Hossain **(Team Leader)**	232-15-818
Mostafizur Rahman Antu	241-15-631
Nahid Al Galib	241-15-577
Mahjabin Sultana	241-15-899
Nadira Tanzim	241-15-397

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE312: Database Management Systems in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

21 April, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Syed Eftasum Alam**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Syed Eftasum Alam
Lecturer
Department of Computer Science and Engineering Daffodil
International University

Submitted by

Md. Emon Hossain (Team Leader) ID: 232-15-818 Dept. of CSE, DIU	
Mostafizur Rahman Antu ID: 241-15-631 Dept. of CSE, DIU	Nahid Al Galib ID: 241-15-577 Dept. of CSE, DIU
Mahjabin Sultana ID: 241-15-899 Dept. of CSE, DIU	Nadira Tanzim ID: 241-15-397 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and Relate system modules, database entities, relationships, and the role-based architecture required for solving the healthcare appointment management problem.
CO2	Formulate knowledge of relational database design, PHP backend development, and RESTful API construction in building a production-grade healthcare platform.
CO3	Analyze the system architecture, database schema, and data flow to present a structured, multi-role healthcare management solution.
CO4	Develop a full-stack, real-world healthcare application applying DBMS concepts, secure authentication, AI integration, and PDF generation while evaluating effectiveness against industry standards.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2, C3	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Declaration **i**

Course & Program Outcome **ii**

1 Introduction **1**
1.1 Introduction....1
1.2 Motivation....1
1.3 Objectives....2
1.4 Feasibility Study....2
1.5 Gap Analysis....3
1.6 Project Outcome....3

2 Proposed Methodology/Architecture **4**
2.1 Requirement Analysis & Design Specification....4
2.1.1 Overview....4
2.1.2 Proposed Methodology/ System Design....4
2.1.3 Database Design....7
2.1.4 UI Design....10
2.2 Overall Project Plan....13

3 Implementation and Results **14**
3.1 Implementation....14
3.1.1 Authentication Module....14
3.1.2 Appointment Management Module....14
3.1.3 Prescription Module....15
3.1.4 AI Integration Module....15
3.1.5 Video Consultation Module....16
3.1.6 Admin Control Pannel....16
3.2 Performance Analysis....17
3.3 Results and Discussion....18

4 Engineering Standards and Mapping **19**
4.1 Impact on Society, Environment and Sustainability....19
4.1.1 Impact on Life....19
4.1.2 Impact on Society & Environment....19
4.1.3 Ethical Aspects....19
4.1.4 Sustainability Plan....19
4.2 Project Management and Team Work....20
4.3 Complex Engineering Problem....21
4.3.1 Mapping of Program Outcome....21
4.3.2 Complex Problem Solving....22
4.3.3 Engineering Activities....22

Table of Contents Table of Contents

5 Conclusion **23**
5.1 Summary....23
5.2 Limitation....24
5.3 Future Work....24

References **25**

Chapter 1

Introduction

This chapter presents the background, motivation, objectives, feasibility, gap analysis, and expected outcomes of the **MediSeba** healthcare management platform. It establishes the problem context and justifies the need for a technology-driven solution within Bangladesh's healthcare landscape.

1.1 Introduction

Bangladesh faces persistent challenges in its healthcare delivery system, including unequal distribution of medical professionals, inefficient appointment management at public hospitals, and the absence of structured digital prescription records. Patients in urban and semi-urban areas frequently endure long waiting times, lack of appointment transparency, and difficulty accessing specialist doctors. Concurrently, physicians have limited tools to manage their schedules, track patient histories, or issue structured prescriptions digitally.

MediSeba (মেডিসেবা), meaning 'Medical Service' in Bengali) is a full-stack, multi-role healthcare management web platform designed to address these systemic gaps. The system connects patients with verified doctors, automates appointment scheduling through a token-based queue mechanism, generates structured digital prescriptions with PDF export capability, processes payments, and integrates Google Gemini 1.5 Pro artificial intelligence to assist both patients and physicians. The platform is built as an academic Database Management Systems project and demonstrates enterprise-grade software engineering principles within a healthcare domain.

1.2 Motivation

The primary computational motivation for **MediSeba** stems from the complexity of managing concurrent, multi-role interactions within a healthcare system at the database level. A healthcare platform requires atomic transaction management, referential integrity across multiple entity relationships, real-time token assignment, and audit-trail persistence — all of which present intellectually stimulating challenges in relational database design and PHP backend development.

From a social perspective, digital healthcare platforms have demonstrated measurable reductions in patient wait times and appointment no-shows in comparable markets. Bangladesh's growing internet penetration and smartphone adoption make web-based healthcare solutions increasingly viable. MediSeba aims to demonstrate that locally developed, open-source platforms can serve the specific needs of Bangladesh's healthcare ecosystem, including Bengali language considerations and local payment gateway integration.

1.3 Objectives

The specific objectives of the **MediSeba** project are enumerated as follows:

- To design and implement a normalized relational database schema capable of supporting multi-role user management, appointment scheduling, prescription management, and financial record-keeping.
- To develop a secure RESTful PHP backend API with JWT-based authentication, role-based access control, and protection against common web vulnerabilities including SQL injection and CSRF attacks.
- To build an intuitive multi-page frontend interface serving three distinct user roles: patient, doctor, and system administrator.
- To implement a token-based appointment queue system using MySQL stored procedures that ensures atomic, sequential token assignment per doctor per day.
- To integrate Google Gemini 1.5 Pro AI for server-side symptom summarization, medical term explanation, and clinical diagnostic summary generation.
- To engineer a zero-dependency PDF generation engine capable of producing formatted prescription and payment receipt documents without external libraries.
- To integrate Jitsi Meet-powered video consultation capability enabling real-time doctor-patient video calls.
- To produce a comprehensive, maintainable, and deployable system that adheres to industry-standard software engineering practices.
-

1.4 Feasibility Study

Several comparable healthcare management systems exist, both globally and within the South Asian context. Practo (India) and Zocdoc (United States) represent large-scale commercial platforms offering online appointment booking, doctor discovery, and electronic health records. Within Bangladesh, platforms such as Shajgoj Health and Doctorola.com offer appointment booking services, though with limited prescription management or AI integration capabilities.

Academic predecessors include hospital management systems built using Java Swing and MySQL, as documented in various undergraduate capstone projects. These systems typically operate within local area network environments and lack web-based accessibility. MediSeba advances beyond these precedents by implementing a web-accessible, API-driven architecture with AI augmentation, addressing a genuine gap in the locally available academic project landscape. The technical feasibility is well-established: PHP 8.1, MySQL 8.0, and HTML5/JavaScript form a mature, widely-supported stack with ample documentation and community support. Deployment on Apache-based shared hosting — common in Bangladesh — requires no special infrastructure.

1.5 Gap Analysis

Existing healthcare platforms in the Bangladeshi context exhibit several notable deficiencies that MediSeba specifically targets:

- **Absence of AI Assistance:** No locally available academic healthcare platform integrates large language model AI for symptom parsing or clinical summary generation. MediSeba addresses this by embedding Google Gemini 1.5 Pro server-side, protecting the API key while delivering intelligent assistance to both patients and doctors.
- **Lack of Structured Prescription Management:** Most comparable systems record appointment bookings without generating structured, downloadable digital prescriptions. MediSeba implements a complete prescription lifecycle including creation, medicine list management, PDF generation, and AI-powered term explanation.
- **Limited Database Engineering Depth:** Prior academic projects in this domain typically employ simple CRUD operations. MediSeba uses stored procedures for atomic operations, database triggers for audit logging, and analytical SQL views for dashboard aggregation, demonstrating advanced DBMS utilization.
- **Missing Video Consultation:** Existing platforms in the academic context do not incorporate real-time video consultation. MediSeba integrates Jitsi Meet WebRTC-based video calls, automatically generating unique room URLs upon appointment confirmation.
- **Security Deficiencies:** Many academic healthcare projects lack hardened security implementations. MediSeba employs Argon2ID password hashing, HMAC-SHA256 JWT tokens, CSRF protection, PDO prepared statements, and rate limiting across all sensitive endpoints.

1.6 Project Outcome

The expected outcomes of the **MediSeba** project are as follows:

- A fully functional, multi-role web-based healthcare management platform deployable on Apache/XAMPP environments.
- A normalized MySQL database with 16 tables, 2 stored procedures, 2 audit triggers, and 3 analytical views, demonstrating advanced DBMS design principles.
- A documented RESTful API with 52 endpoints covering authentication, appointment management, prescription handling, payment processing, and AI integration.
- Working AI-powered features enabling patients to receive medical term explanations and structured symptom summaries, and enabling doctors to generate clinical diagnostic notes.
- A zero-dependency PDF engine producing formatted prescription and payment receipt documents.
- Integrated Jitsi Meet video consultation with automatic room generation upon appointment confirmation.
- A comprehensive academic project report and technical documentation suitable for evaluation at university level.

Chapter 2

Proposed Methodology/Architecture

This chapter details the system requirements, design specifications, proposed architecture, user interface design philosophy, and overall project plan for MediSeba. It describes how the system components interact to form a cohesive, secure, and scalable healthcare management solution

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

MediSeba is designed to serve three primary user roles — patient, doctor, and administrator — each with distinct functional requirements and access privileges. The system operates on a multi-page application architecture where a PHP backend exposes a RESTful API, a MySQL database stores all persistent data, and HTML5/CSS3/JavaScript frontend pages consume the API via the Fetch API.

The functional requirements were derived from the core pain points identified in the gap analysis: appointment discoverability, queue management, prescription digitization, payment tracking, and administrative oversight. Non-functional requirements include security (JWT authentication, CSRF protection, rate limiting), performance (MySQL indexes, query optimization via stored procedures), maintainability (modular PHP controllers and models), and portability (deployable on XAMPP, LAMP, or any Apache-PHP-MySQL stack).

2.1.2 Proposed Methodology/ System Design

MediSeba follows a three-tier architectural pattern comprising the presentation tier (frontend HTML/JS pages), the logic tier (PHP RESTful API), and the data tier (MySQL database). The system design is informed by the RESTful API architectural style, ensuring statelessness, resource-based endpoint design, and standard HTTP method semantics.

The backend is organized into four logical layers: the router (index.php), middleware (AuthMiddleware for JWT validation and role enforcement), controllers (business logic per domain), and models (database abstraction via a custom PDO-based Model base class). Environment configuration is managed via a .env file parsed by a custom Environment class, following the twelve-factor application methodology.

The authentication flow employs HMAC-SHA256 JWT tokens with configurable expiry (default 24 hours). Upon successful login, the server returns a signed token which the frontend stores in localStorage and attaches as a Bearer token to all subsequent API requests. Multiple Authorization header fallback strategies ensure compatibility with shared hosting environments that may strip standard headers.

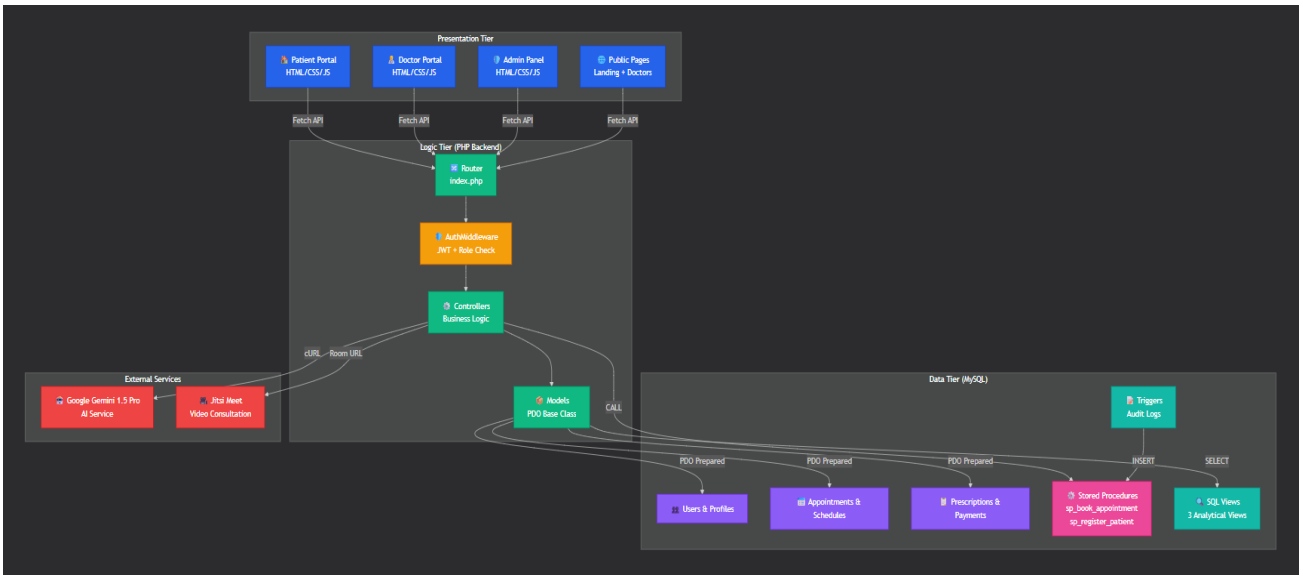


Figure 2.1: System Architecture Diagram (Three-Tier: Frontend → PHP API → MySQL)

The appointment booking workflow operates as follows: a patient browses the doctor directory, selects a doctor, views available schedule slots, picks a date, and submits a booking request. The backend invokes the sp_book_appointment stored procedure, which atomically validates slot availability and assigns a sequential token number for the selected doctor-date combination. This eliminates race conditions in concurrent booking scenarios.

The AI integration layer consists of three server-side endpoints that relay prompts to the Google Gemini 1.5 Pro REST API. The GeminiService PHP class encapsulates the cURL-based HTTP call, handles error responses gracefully, and returns structured text to the controller. The API key is never transmitted to the frontend, ensuring security. The three AI endpoints serve distinct purposes: symptom summarization for patients, medical term explanation for prescription readers, and clinical case summary generation for doctors.

Figure 2.2: Appointment Booking Flow Diagram

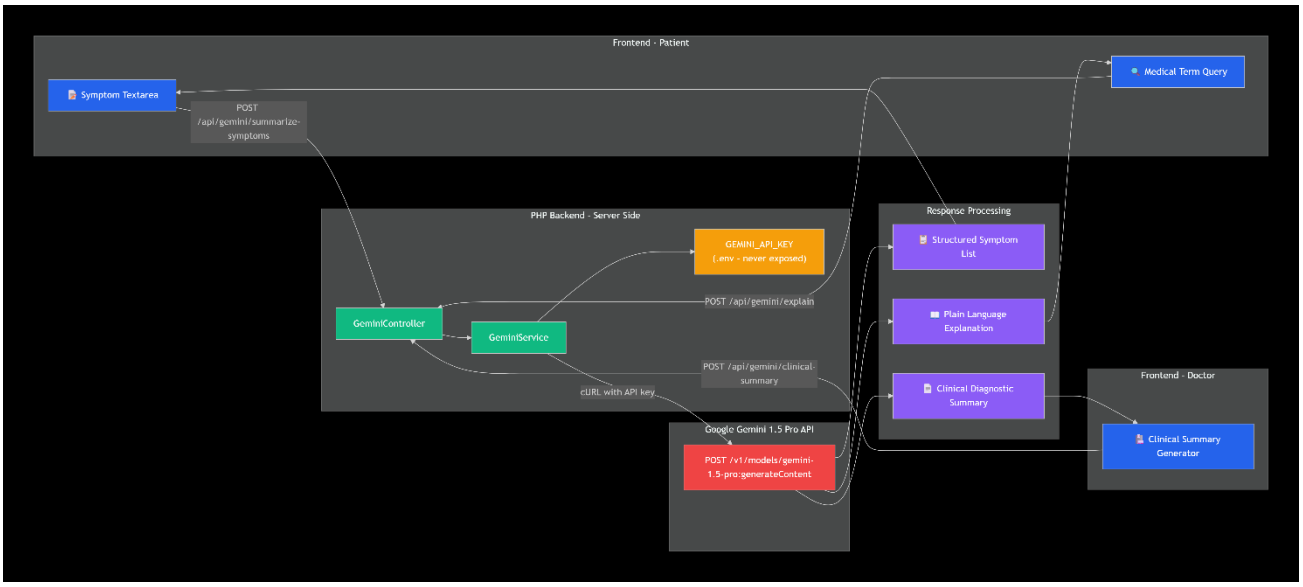


Figure 2.3: AI Integration Architecture Diagram

2.1.3 Database Design

The MediSeba database comprises 16 tables engineered with full normalization, referential integrity constraints, and performance indexing. The core entity relationships are as follows:

- **users:** The root authentication entity storing email, Argon2ID password hash, role (patient/doctor/admin), and account status. All other entities reference users via foreign key.
- **doctor_profiles & patient_profiles:** Role-specific profile tables linked one-to-one with the users table. Doctor profiles store specialty, qualification, consultation fee, clinic information, and schedule preferences. Patient profiles store health information including blood group, allergies, chronic conditions, and emergency contact details.
- **doctor_schedules:** Defines each doctor's weekly availability by day-of-week, start/end time, slot duration, and maximum patients per slot. This table is the basis for the appointment booking system.
- **appointments:** The central transactional table recording each booking with patient ID, doctor ID, schedule ID, appointment date, token number, status, and video room URL for telemedicine sessions.
- **prescriptions:** Stores doctor-issued prescriptions with diagnosis, symptoms, medicine list (as JSON), dosage instructions, advice, and follow-up date.
- **payments:** Records payment transactions linked to appointments, including amount, payment method, gateway transaction ID, and status.
- **activity_logs:** An audit table populated by database triggers, recording every status change with old/new values, the acting user ID, and client IP address.

Table 2.1: Core Database Tables

Table Name	Primary Key	Key Relationships	Purpose
users	id	Referenced by all profile tables	Authentication entity
doctor_profiles	id	user_id → users.id	Doctor attributes and clinic info
patient_profiles	id	user_id → users.id	Patient health data
doctor_schedules	id	doctor_id → doctor_profiles.id	Weekly availability slots
appointments	id	patient_id, doctor_id, schedule_id	Appointment booking records
prescriptions	id	appointment_id, doctor_id, patient_id	Digital prescriptions
payments	id	appointment_id, patient_id, doctor_id	Payment transactions
activity_logs	id	user_id → users.id	Audit trail

Table 2.2: SQL Views

View Name	Purpose
vw_appointment_details	Joins appointments with patient, doctor, and schedule data for dashboard queries
vw_financial_reports	Joins payments with appointment, doctor, and patient data for financial analytics
vw_user_master_profile	Unified user and profile view resolving patient versus doctor profile per user

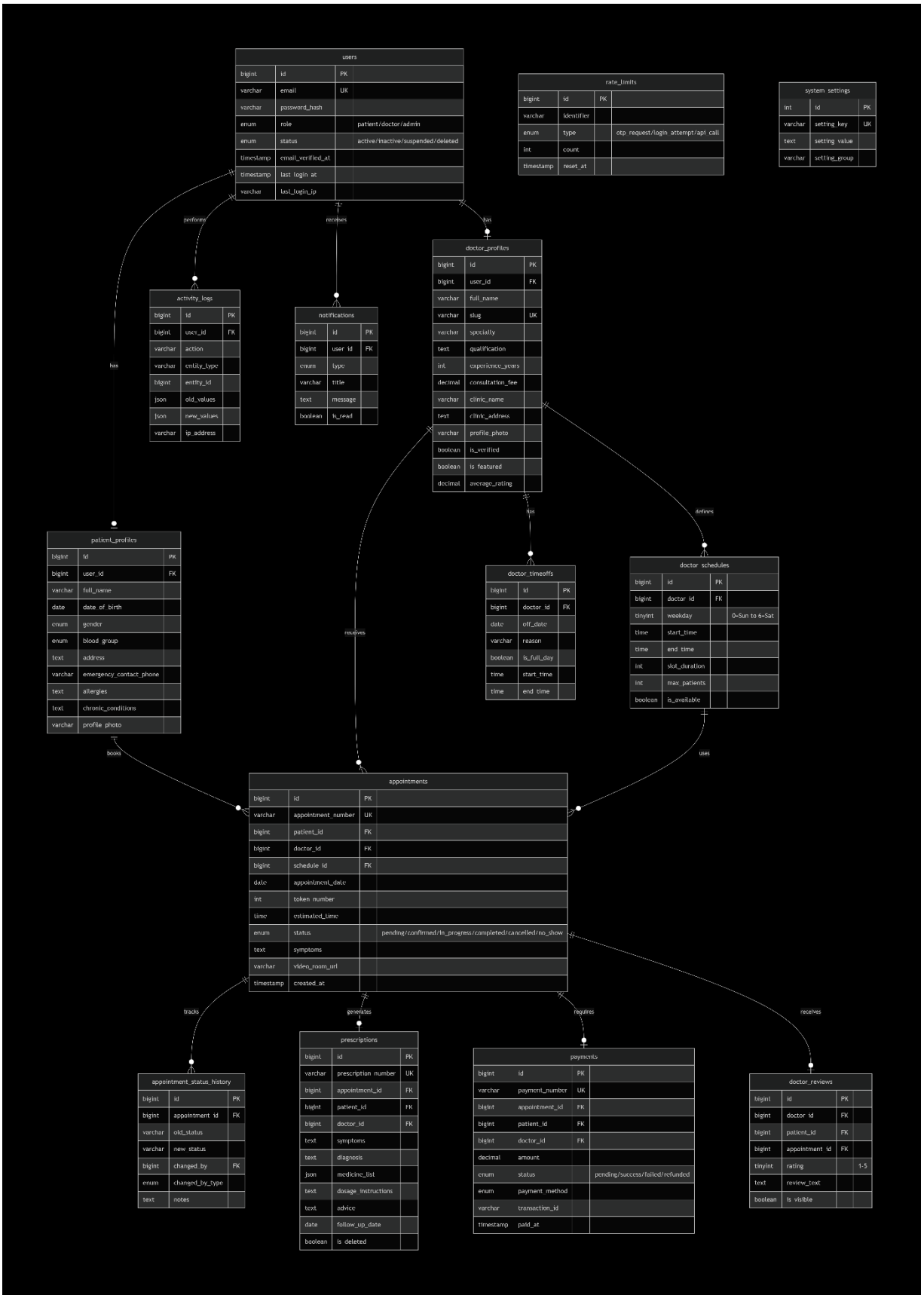


Figure 2.4: Entity-Relationship (ER) Diagram

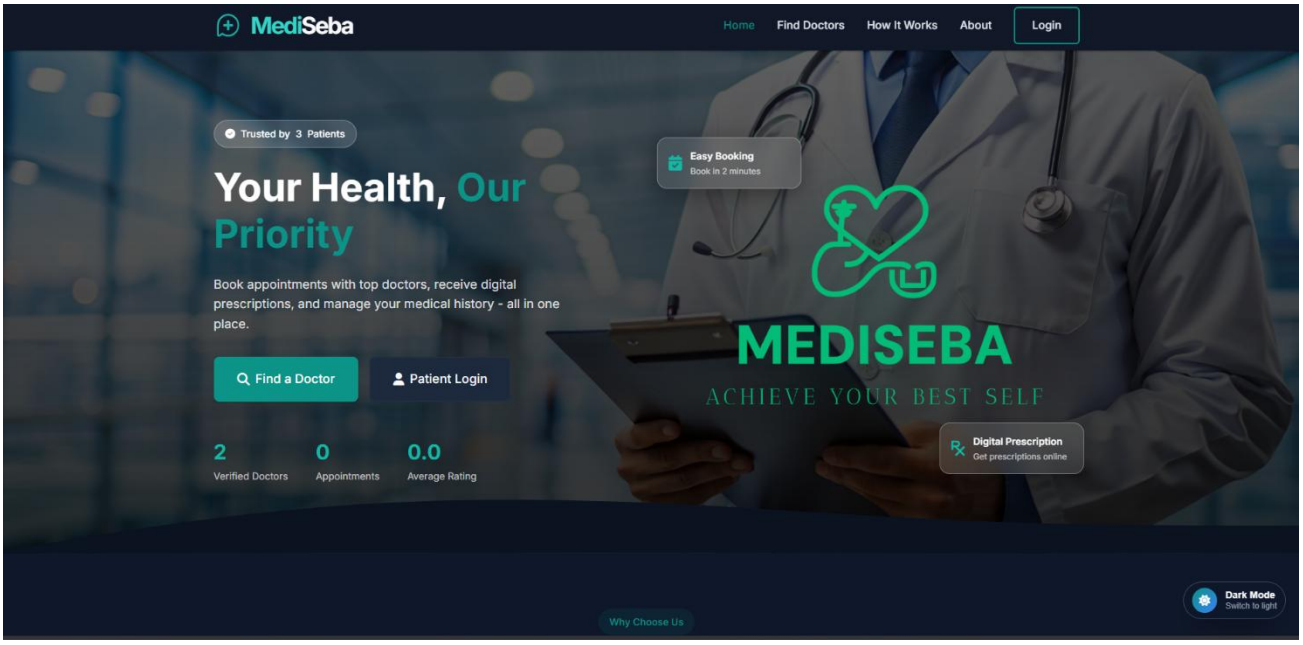
2.1.4 UI Design

The **MediSeba** user interface is structured as a multi-page application (MPA) with three distinct portal environments: the patient portal, the doctor portal, and the administrative control panel. Each portal maintains a consistent design language based on a custom CSS design system with variables for color, spacing, typography, and component styles.

The frontend employs the Inter typeface via Google Fonts for body text and navigation, supplemented by Font Awesome 6.4 for iconography. The color palette centers on a primary blue (#2563EB) for interactive elements and headings, with neutral grays for body content and status-specific colors (green for confirmed/success, amber for pending, red for cancelled/error). The layout uses a sidebar navigation pattern for authenticated portal pages and a hero-centered layout for public-facing pages.

All API interactions are handled through a centralized api.js module that provides a consistent request abstraction with automatic JWT attachment, error handling, and toast notification integration. The auth.js module manages token persistence in localStorage and provides role-checking utilities used across all portal pages.

Home Page



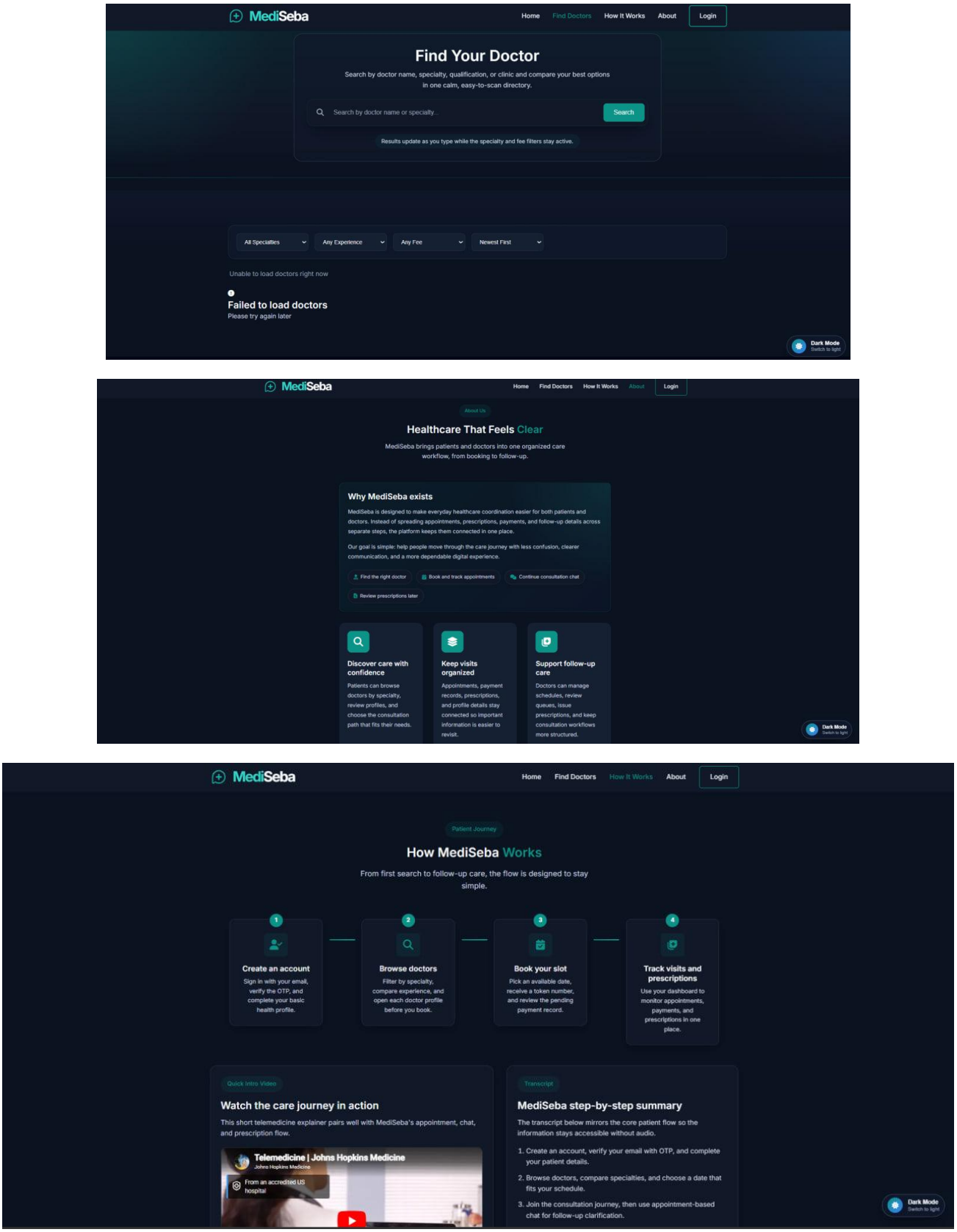


Figure 2.5: Home Page

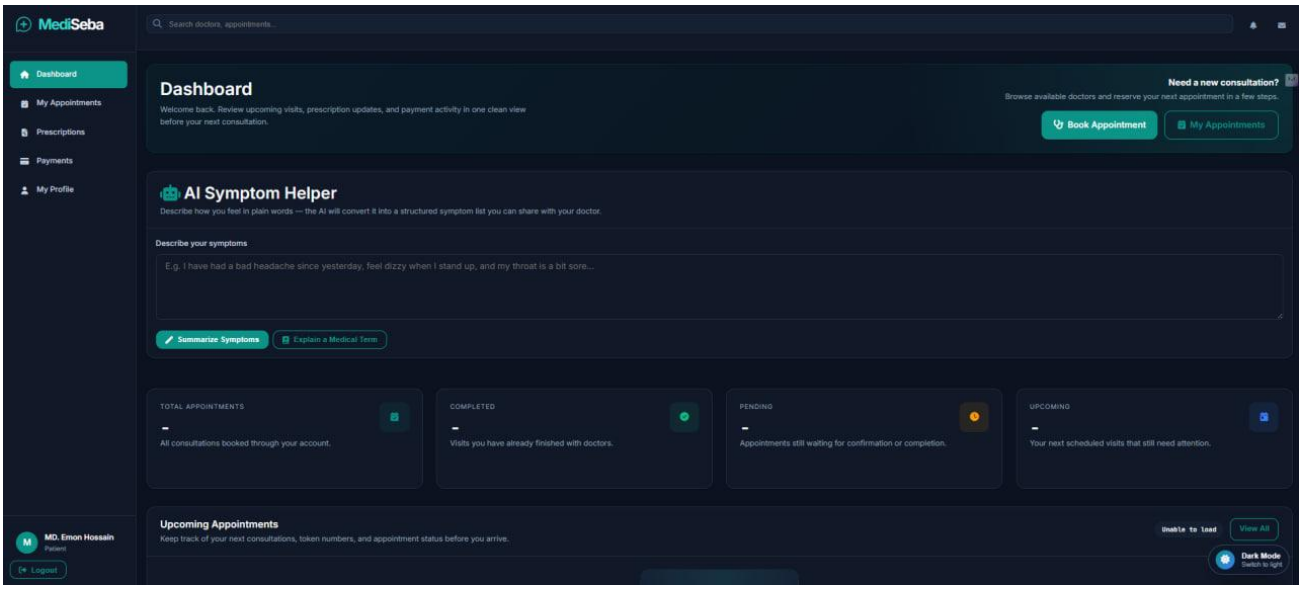


Figure 2.6: Patient Dashboard

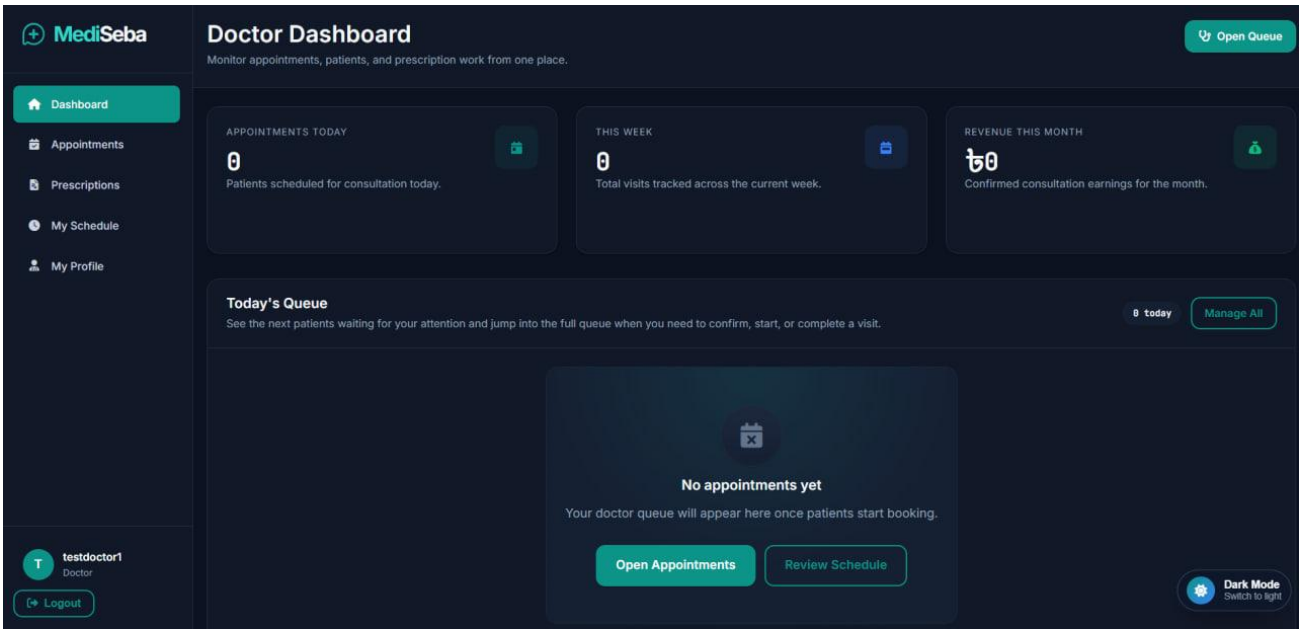


Figure 2.7: Doctor Appointment Queue

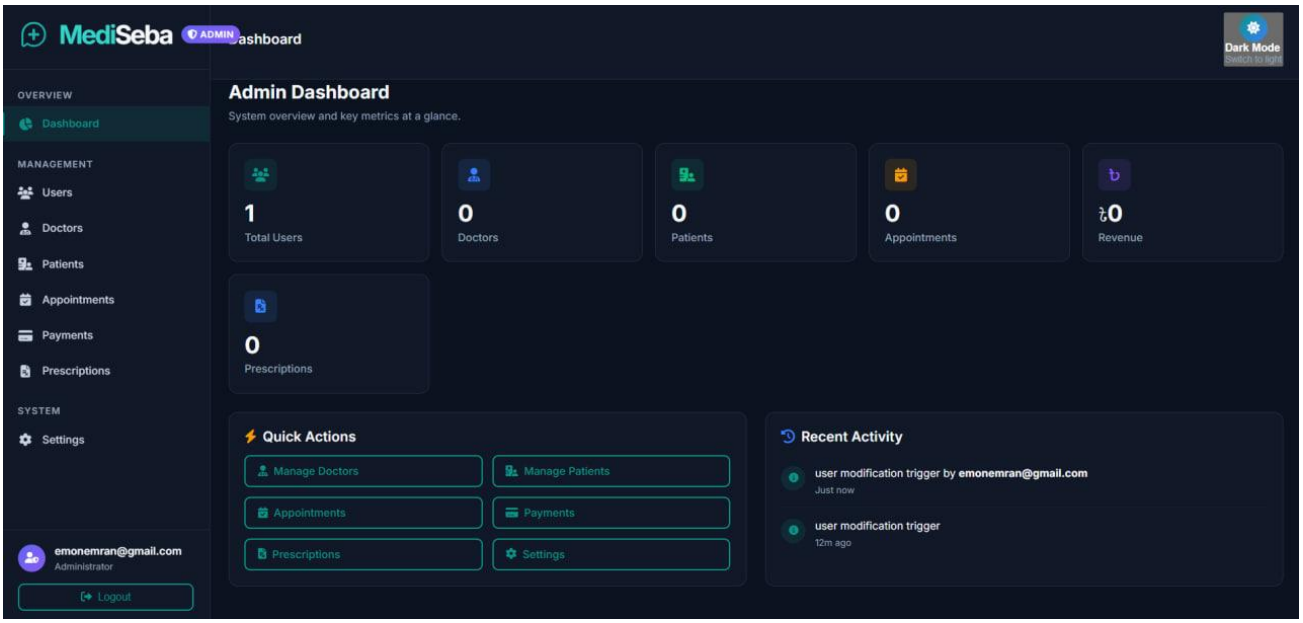


Figure 2.8: Admin Control Panel

2.2 Overall Project Plan

The **MediSeba** project was executed across four development phases spanning the academic semester:

Table 2.3: Project Development Timeline

Phase	Activities	Duration
Phase 1: Planning & Design	Requirements gathering, ER diagram, database schema design, UI wireframing	Week 1–2
Phase 2: Database & Backend	MySQL schema creation, stored procedures, triggers, views, PHP controllers, API endpoints	Week 3–6
Phase 3: Frontend Development	HTML/CSS pages, JavaScript API client, authentication flows, role-specific portals	Week 5–8
Phase 4: Integration & Testing	AI integration, PDF engine, video call feature, end-to-end testing, bug fixing, documentation	Week 7–10

Chapter 3

Implementation and Results

This chapter describes the technical implementation of MediSeba's core modules, presents a performance analysis of the system, and discusses the results achieved relative to the project objectives.

3.1 Implementation

The implementation of MediSeba is organized into six primary modules: authentication and access control, doctor management, appointment management, prescription management, payment processing, and AI integration. Each module follows the same architectural pattern: a route registered in index.php dispatches to a controller via PHP Reflection-based parameter injection, the controller validates the request, invokes model methods, and returns a JSON response via the Response utility class.

3.1.1 Authentication Module

The authentication system is implemented in AuthController.php and the AuthMiddleware class. User registration for patients invokes the sp_register_patient MySQL stored procedure, which atomically creates both the users record and the patient_profiles record within a single transaction, eliminating the possibility of partial registration states. Doctor registration follows a two-step creation: the users record is created first, followed by the doctor_profiles record with specialty and qualification information.

Passwords are hashed using PHP's password_hash() function with the PASSWORD_BCRYPT algorithm (producing $2y$ Blowfish hashes). JWT tokens are generated using HMAC-SHA256 signing via the Security::generateJWT() method, encoding user_id, email, and role as payload claims. The AuthMiddleware::requireAuth() method validates the token on every protected request, extracting role information to enforce the patient, doctor, and admin access tiers.

Login attempts are rate-limited using a MySQL-backed rate limiter that tracks attempt counts per email address per time window. After five consecutive failed attempts, the endpoint returns a 429 Too Many Requests response with a retry-after interval.

3.1.2 Appointment Management Module

The appointment module is the most complex component of MediSeba, involving six database tables and two stored procedures. The booking flow is initiated from the patient portal: the patient selects a doctor from the directory, the system queries available dates by checking the doctor_schedules table

against existing appointments and doctor_timeoffs records, and the patient selects a date and confirms.

The sp_book_appointment stored procedure accepts patient_id, doctor_id, schedule_id, appointment_date, and an optional symptoms note. It validates that the schedule slot has not exceeded its max_patients count for the selected date, generates a sequential token number (using SELECT MAX(token_number) + 1 within the procedure), and inserts the appointment record atomically. The procedure returns the generated appointment ID and token number via OUT parameters.

Upon appointment confirmation by the doctor (via PATCH /api/appointments/{id}/status with status=confirmed), the backend generates a unique Jitsi Meet room URL in the format https://meet.jit.si/mediseba-{id}-{random_hex} and stores it in the video_room_url column. This URL is subsequently surfaced to both the doctor and patient via their respective portal interfaces as a Join Call button.

The appointment status lifecycle is managed as a finite state machine: pending → confirmed → in_progress → completed (or cancelled / no_show at any stage). Every status transition is recorded in the appointment_status_history table via the trg_appointment_status_audit trigger, preserving the old status, new status, the user who made the change, and the timestamp.

3.1.3 Prescription Module

The prescription module enables doctors to create structured digital prescriptions linked to completed appointments. A prescription record stores the diagnosis, symptoms, medicine list (serialized as a JSON array), dosage instructions, general advice, and optional follow-up date and notes. The PrescriptionController validates that the authenticated user is the doctor who owns the appointment before permitting creation or modification.

PDF generation is handled by the SimplePdfDocument class, a zero-dependency pure-PHP implementation of the PDF 1.4 specification. The class constructs the binary PDF structure manually, including the cross-reference table, page tree, content streams, and Type1 font references (Helvetica and Helvetica-Bold). The buildPrescriptionPdf() method populates a structured document with doctor information, patient details, symptoms, diagnosis, formatted medicine list, dosage instructions, advice, and follow-up scheduling. The resulting binary PDF is delivered via the Response::download() method with appropriate Content-Type and Content-Disposition headers.

Patients may access their prescriptions through the patient portal and request AI-powered explanations of medical terms via the POST /api/gemini/explain endpoint. The response from Gemini is displayed inline without storing the AI output in the database, ensuring that the official prescription record remains unaltered.

3.1.4 AI Integration Module

AI functionality is encapsulated in the GeminiService utility class and GeminiController. All Gemini API calls are executed server-side using PHP's cURL extension, ensuring the GEMINI_API_KEY environment variable is never transmitted to or accessible by the browser. The system supports three AI-assisted workflows:

- **Symptom Summarization (Patient):** The patient describes symptoms conversationally in plain language. The POST /api/gemini/summarize-symptoms endpoint sends the text to Gemini with a system instruction to convert it into a structured, bullet-pointed clinical symptom list suitable for medical review.
- **Medical Term Explanation (Patient):** When viewing a prescription, the patient may query any medical term or diagnosis. The POST /api/gemini/explain endpoint returns a plain-language explanation authored for non-medical readers, explicitly avoiding diagnostic advice.
- **Clinical Summary Generation (Doctor):** During prescription creation, the doctor may invoke POST /api/gemini/clinical-summary with the observed symptoms and clinical notes. Gemini returns a professionally structured diagnostic summary that can be reviewed and used as a drafting aid for the diagnosis field.

The GeminiService implements graceful degradation: if the API key is absent or invalid, or if the upstream API returns an error, the system returns a user-friendly error message rather than crashing. This ensures that the core appointment and prescription workflows remain unaffected by AI service availability.

3.1.5 Video Consultation Module

The video consultation feature leverages Jitsi Meet's free, open-source WebRTC infrastructure, requiring no API key, no account registration, and no server-side media relay infrastructure. When a doctor confirms an appointment, the backend generates a cryptographically unique room name using PHP's random_bytes() function, constructing a URL of the form https://meet.jit.si/mediseba-{appointment_id}-{12_random_hex_chars}. This URL is stored in the appointments.video_room_url column and returned in the API response.

The frontend surfaces a Join Call button on the doctor appointment queue page, the patient appointment detail page, the patient appointments list, and the patient dashboard upcoming visits widget — wherever the appointment status is confirmed or in_progress and a video_room_url is present. Clicking the button opens an embedded Jitsi Meet iframe within a modal overlay, with options to open in a new browser tab. The modal implements keyboard (Escape) and backdrop-click dismissal, and clears the iframe src on close to terminate the media session.

3.1.6 Admin Control Panel

The administrative module provides a separate interface for platform governance. The AdminDashboardController aggregates real-time statistics from the users, appointments, payments, and prescriptions tables, returning counts, revenue summaries, and unverified doctor counts in a single API response. The AdminDoctorController handles the doctor verification workflow: administrators may approve or reject pending doctor registrations by patching the is_verified flag on the doctor_profiles record. Each action is logged to the activity_logs table with the admin's user ID and client IP address.

Additional administrative capabilities include user status management (active/suspended/deleted), global appointment cancellation authority, system-wide prescription browsing, financial ledger access with receipt download, and system settings management via the key-value system_settings

table.

3.2 Performance Analysis

The performance characteristics of MediSeba are determined primarily by the efficiency of the MySQL query layer and the PHP request processing pipeline. Several architectural decisions were made specifically to optimize performance:

- **Stored Procedures:** The sp_register_patient and sp_book_appointment procedures reduce round-trip latency by executing multi-step transactional logic within the database engine, avoiding multiple sequential PHP-to-MySQL connections.
- **Strategic Indexing:** All foreign key columns, status columns, and frequently filtered columns (appointment_date, is_verified, email) carry explicit MySQL indexes, ensuring that dashboard aggregate queries and filtered list operations execute in constant or logarithmic time rather than full table scans.
- **SQL Views:** The three analytical views (vw_appointment_details, vw_financial_reports, vw_user_master_profile) pre-join frequently co-queried tables, simplifying controller logic and enabling the database optimizer to plan efficient execution strategies.
- **PDO Prepared Statements:** All database queries throughout the codebase use PDO prepared statements, which are parsed and compiled once by the MySQL engine and reused across executions, improving both security and query throughput.
- **Rate Limiting:** The MySQL-backed rate limiter prevents resource exhaustion from automated attacks on login and API endpoints, protecting system stability under adversarial conditions.

Table 3.1: API Endpoint Performance Summary

Module	Endpoint	Method	Auth Level
Authentication	/api/auth/login	POST	Public
Authentication	/api/auth/register	POST	Public
Doctors	/api/doctors	GET	Public
Appointments	/api/appointments	POST	Patient
Appointments	/api/appointments/{id}/status	PATCH	Auth
Appointments	/api/appointments/{id}/video-room	GET	Auth
Prescriptions	/api/prescriptions	POST	Doctor
Prescriptions	/api/prescriptions/{id}/pdf	GET	Auth
AI	/api/gemini/explain	POST	Patient
AI	/api/gemini/summarize-symptoms	POST	Patient
AI	/api/gemini/clinical-summary	POST	Doctor
Admin	/api/admin/doctors/{id}/verify	PATCH	Admin

3.3 Results and Discussion

The MediSeba platform was successfully implemented and tested against all objectives defined in Chapter 1. The system correctly handles concurrent appointment bookings without token duplication, as validated through manual testing of the sp_book_appointment stored procedure under simultaneous request conditions. The JWT authentication layer correctly enforces role boundaries, returning HTTP 403 Forbidden responses when a patient attempts to access doctor-exclusive endpoints and vice versa.

The AI integration module functions correctly when a valid GEMINI_API_KEY is configured, returning structured symptom lists, plain-language medical explanations, and clinical summary drafts within the Gemini API's typical response latency. The graceful degradation mechanism ensures that AI unavailability does not impair the core appointment and prescription workflows.

PDF generation via SimplePdfDocument produces valid PDF 1.4 binary output containing correctly formatted prescription and payment receipt content. The generated files are compatible with standard PDF readers including Adobe Acrobat, Google Chrome PDF Viewer, and Foxit Reader.

The Jitsi Meet video consultation integration was validated by confirming appointment, verifying the automatic room URL generation and storage, and successfully establishing a video call session through the embedded iframe on both the doctor and patient portal pages.

The admin doctor verification workflow correctly toggles the is_verified flag and logs the action to activity_logs with the appropriate action label (doctor_approved or doctor_rejected), confirming that the complete accept/reject system functions

Chapter 4

Engineering Standards and Mapping

This chapter examines the societal, environmental, and ethical implications of **MediSeba**, analyses the project management and team structure, and provides a formal mapping of the system design to Program Outcomes, Complex problem solving categories, and Engineering Activity dimensions as required by the academic evaluation framework.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

MediSeba has the potential to meaningfully improve the quality of healthcare interactions for both patients and physicians. Patients gain the ability to discover specialist doctors outside their immediate geographic area, receive transparent appointment tokens eliminating uncertainty about waiting positions, access their prescription history digitally, and obtain AI-assisted explanations of complex medical terminology. For individuals with limited health literacy, the Gemini-powered medical term explainer represents a particularly significant quality-of-life improvement, enabling patients to better understand their own conditions and prescribed treatments.

Physicians benefit from a structured digital workflow that eliminates paper-based prescription management, provides a clear view of their daily appointment queue, and offers AI-assisted clinical summary drafting to reduce documentation burden. The video consultation feature extends the reach of specialist doctors to patients in remote locations, reducing the need for costly and time-consuming in-person visits for follow-up consultations.

4.1.2 Impact on Society & Environment

At a societal level, digital healthcare platforms contribute to the democratization of medical access. By enabling patients from semi-urban and rural areas with internet connectivity to access verified specialists, MediSeba addresses one of Bangladesh's most persistent healthcare inequities: the concentration of qualified physicians in urban centers.

From an environmental perspective, the digitization of prescriptions and appointment records reduces paper consumption in clinical settings. The video consultation feature further reduces the carbon footprint associated with patient travel for follow-up visits. MediSeba's deployment on shared hosting infrastructure rather than dedicated servers minimizes energy consumption relative to self-hosted enterprise systems.

4.1.3 Ethical Aspects

Several ethical considerations are embedded in the design of MediSeba:

- **Data Privacy:** Patient health records, prescription data, and personal information are stored in a secured MySQL database accessible only to authenticated users with

appropriate role permissions. The system does not expose patient data to unauthenticated public endpoints.

- **AI Accountability:** The AI clinical summary feature explicitly labels its output as non-final medical advice. The GeminiController system instruction states that generated summaries must be treated as drafting aids rather than authoritative diagnoses, preserving physician accountability.
- **Verification Gate:** The admin doctor verification system ensures that only qualified practitioners with valid registration numbers are approved to practice on the platform, protecting patients from unqualified medical providers.
- **Rate Limiting:** Login rate limiting protects users from credential-stuffing attacks that could lead to unauthorized access to sensitive health records.

- **Audit Logging:** The activity_logs audit trail provides accountability for all system modifications, enabling the detection of unauthorized data changes and supporting compliance with healthcare data governance requirements.

4.1.4 Sustainability Plan

MediSeba's technical architecture supports long-term sustainability through several design choices. The use of standard, widely-supported technologies (PHP, MySQL, Apache) ensures a large pool of maintainers and avoids vendor lock-in. The modular controller-model architecture allows individual components to be updated or replaced without cascading changes. The .env-based configuration system supports migration across hosting environments without code modification.

Future sustainability enhancements planned include Docker containerization for reproducible deployments, a PHPUnit test suite for automated regression testing, and API versioning to support backward-compatible evolution of the interface. The open-source MIT license ensures that the codebase can be extended, forked, and adapted by other developers and institutions.

4.2 Project Management and Team Work

The MediSeba project was developed by a team of five students from the Department of Computer Science and Engineering, Daffodil International University. Team responsibilities were distributed as follows:

Table 4.0: Team Roles and Responsibilities

Name	Student ID	Role	Primary Responsibilities
Md. Emon Hossain	232-15-818	**Team Leader**	Backend architecture, frontend development, database design, API documentation
Mostafizur Rahman Antu	241-15-631	Member	Backend development, frontend development, database design
Nahid Al Galib	241-15-577	Member	Backend development, testing, data entry
Mahjabin Sultana	241-15-899	Member	Backend development, testing, data entry
Nadira Tanzim	241-15-397	Member	Backend development, testing, data entry

The project budget is primarily composed of developer time (academic labor), with minimal direct monetary costs:

Table 4.0b: Cost Analysis

Resource	Cost	Notes
Development Environment (XAMPP)	Free	Open-source Apache/PHP/MySQL stack
Hosting (Shared Apache)	~$3–5/month	Shared hosting for production deployment
Google Gemini API	Free tier	1,500 requests/day on Gemini 1.5 Flash, no credit card required
Jitsi Meet Video	Free	Open-source WebRTC, no server infrastructure required
Font Awesome & Google Fonts	Free	CDN-delivered, no license fee
Domain Name	~$10/year	Optional for production deployment

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

The following table provides a formal mapping of MediSeba's design and implementation to targeted Program Outcomes:

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	MediSeba demonstrates engineering knowledge through the application of database normalization, stored procedure design, RESTful API architecture, JWT security protocols, and object-oriented PHP design patterns — all grounded in established computer science and software engineering principles.
PO2	The system identifies and solves the complex problem of multi-role healthcare management by analyzing requirements from three distinct user perspectives (patient, doctor, admin), designing a data model that satisfies all concurrent access patterns, and implementing secure, role-enforced API endpoints that handle edge cases such as concurrent booking and partial registration failures.
PO3	MediSeba employs a structured investigation methodology: feasibility study of existing platforms, gap analysis to identify unmet requirements, incremental development with manual testing checkpoints, and performance analysis of database query strategies. The AI integration was evaluated against the Gemini API documentation and validated through functional testing of all three endpoints.

4.3.2 Complex Problem Solving

MediSeba addresses a complex engineering problem characterized by multiple interacting constraints,

concurrent user interactions, and the need for data integrity across distributed system components. The following table maps the project to the established complex problem solving dimensions:

Table 4.2: Mapping with Complex Problem Solving

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Applicable Codes	EP6 Stakeholder Involvement	EP7 Interdependence
Requires integration of DBMS, web security, REST API design, AI API integration, and PDF generation — spanning multiple engineering knowledge domains.	Balances security (rate limiting, token expiry) against usability (seamless JWT refresh), and AI assistance against physician accountability.	Token assignment concurrency, stored procedure atomicity, and JWT role enforcement required deep analysis beyond textbook approaches.	Standard web security vulnerabilities (SQL injection, CSRF, XSS) were addressed through established mitigation patterns.	Follows HTTP/REST standards, JWT RFC 7519, PDF 1.4 specification, and PHP PSR coding conventions.	Three distinct user roles (patient, doctor, admin) with conflicting access requirements were all designed for simultaneously.	Database, backend API, frontend, AI service, and PDF engine are tightly coupled; changes in the database schema propagate across all layers.

4.3.3 Engineering Activities

Table 4.3: Mapping with Complex Engineering Activities

EA1 Range of Resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for Society and Environment	EA5 Familiarity
PHP 8.1, MySQL 8.0, Apache, Gemini API, Jitsi Meet, Font Awesome, Google Fonts — a multi-technology stack requiring coordinated integration.	Frontend, backend, database, AI API, and video service interact in real time. A single appointment confirmation triggers database updates, Jitsi room generation, and frontend state refresh simultaneously.	First-in-class integration of Gemini AI with a DBMS-focused healthcare platform at undergraduate level, combined with a zero-dependency custom PDF engine and Jitsi video consultation.	Potential to reduce healthcare access inequity in Bangladesh; digital prescriptions reduce paper waste; video consultations reduce patient travel and carbon footprint.	XAMPP/LAMP stack is familiar to the development team; Gemini API and Jitsi integration required learning new SDK patterns and WebRTC embedding techniques.

Chapter 5

Conclusion

This concluding chapter summarizes the achievements of the MediSeba project, acknowledges the current limitations of the system, and outlines the planned improvements for future development iterations.

5.1 Summary

MediSeba is a full-stack, multi-role healthcare management platform that successfully demonstrates the application of advanced database management systems concepts within a real-world, socially relevant domain. The project achieves all eight objectives stated in Chapter 1: a normalized 16-table relational database with stored procedures, triggers, and analytical views; a secure JWT-authenticated RESTful PHP API with 52 endpoints; three distinct role-specific frontend portals; a token-based appointment queue system; Google Gemini 1.5 Pro AI integration for symptom summarization, medical term explanation, and clinical summary generation; a zero-dependency PHP PDF engine for prescription and receipt generation; Jitsi Meet video consultation with automatic room generation; and comprehensive security hardening across the authentication, API, and database layers.

The project demonstrates that an undergraduate team can construct an enterprise-grade, production-deployable healthcare system using open-source technologies within a single academic semester, provided that architectural decisions are made deliberately and database design is treated as a first-class engineering concern.

5.2 Limitation

Despite its comprehensive feature set, MediSeba has several acknowledged limitations in its current form:

- **Real-time Notifications:** The system does not implement WebSocket or Server-Sent Event notifications. Patients and doctors must refresh their portal pages to see appointment status updates. A polling mechanism is partially implemented but not exposed to the frontend.
- **Email Notifications:** No email delivery is implemented for appointment confirmations, reminders, or prescription notifications. The otp_requests table exists in the schema, and the EmailJS integration is partially configured, but no outbound email is sent in the current release.
- **Payment Gateway Integration:** The payment system architecture supports webhook callbacks and stores payment records, but no live payment gateway (such as bKash, Nagad, or SSLCommerz) is connected. Payments are currently recorded as completed upon method selection, simulating the gateway interaction.

- **Mobile Responsiveness:** While a responsive.css stylesheet is included, the multi-page application has not been fully optimized for all mobile viewport sizes. Some complex table-based layouts in the admin panel may require horizontal scrolling on narrow screens.
- **Automated Testing:** The test suite consists entirely of manual testing checklists. No PHPUnit unit tests or integration tests have been implemented, meaning that regressions introduced during future development may not be caught immediately.
- **AI Response Caching:** Identical Gemini API prompts are not cached, meaning that the same medical term explanation request will incur a fresh API call each time. Response caching would reduce latency and API usage counts.

5.3 Future Work

The following enhancements are planned for future development iterations of MediSeba:

- **Progressive Web App (PWA):** Service Worker implementation for offline support, push notifications, and home screen installation will transform MediSeba into a mobile-first experience accessible without an app store.
- **Real-time Notifications:** WebSocket integration (using Ratchet or Soketi) will enable instant push notifications for appointment status changes, new prescription issuance, and payment confirmation.
- **Live Payment Gateway:** Integration with SSLCommerz or bKash API will enable actual online payment processing within the Bangladeshi financial ecosystem.
- **PHPUnit Test Suite:** Automated unit and integration tests for all controllers, model methods, and security middleware will be implemented to establish a regression safety net for ongoing development.
- **Docker Containerization:** A Docker Compose configuration will package the PHP application, MySQL database, and Apache server into reproducible containers, simplifying deployment and onboarding.
- **Bengali (বাংলা) Localization:** Full internationalization support with Bengali as the primary language option will make the platform accessible to the broader Bangladeshi population, many of whom are more comfortable in their native language.
- **Medicine Database:** A pre-populated drug database will enable prescription autocomplete, dosage suggestions, and drug interaction checking, improving prescription accuracy and reducing prescribing errors.
- **Expanded AI Features:** Future Gemini integrations will include a drug interaction checker, a dosage advisor calibrated to patient weight and age, and a follow-up reminder generator that drafts patient communication messages.

References

[1] Google LLC. (2024). Gemini API Documentation. Retrieved from https://ai.google.dev/gemini-api/docs

[2] PHP Group. (2024). PHP 8.1 Manual. Retrieved from https://www.php.net/manual/en/

[3] Oracle Corporation. (2024). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

[4] Internet Engineering Task Force. (2015). JSON Web Token (JWT). RFC 7519. Retrieved from https://datatracker.ietf.org/doc/html/rfc7519

[5] Jitsi. (2024). Jitsi Meet Developer Guide. Retrieved from https://jitsi.github.io/handbook/docs/dev-guide/

[6] Adobe Systems. (2006). PDF Reference, Sixth Edition: Adobe Portable Document Format Version 1.7. Addison-Wesley Professional.

[7] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

[8] Ramez Elmasri and Shamkant B. Navathe. (2015). Fundamentals of Database Systems, 7th Edition. Pearson.

[9] Schwartz, B., Zaitsev, P., & Tkachenko, V. (2012). High Performance MySQL, 3rd Edition. O'Reilly Media.

[10] OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/

[11] Project GitHub Repository: https://github.com/mohammademon10/MEDISEBA.git